ARGO CD GITOPS › MY LEARNING NOTES › SESSION 01

Screenshots: Argo CD Components & Architecture

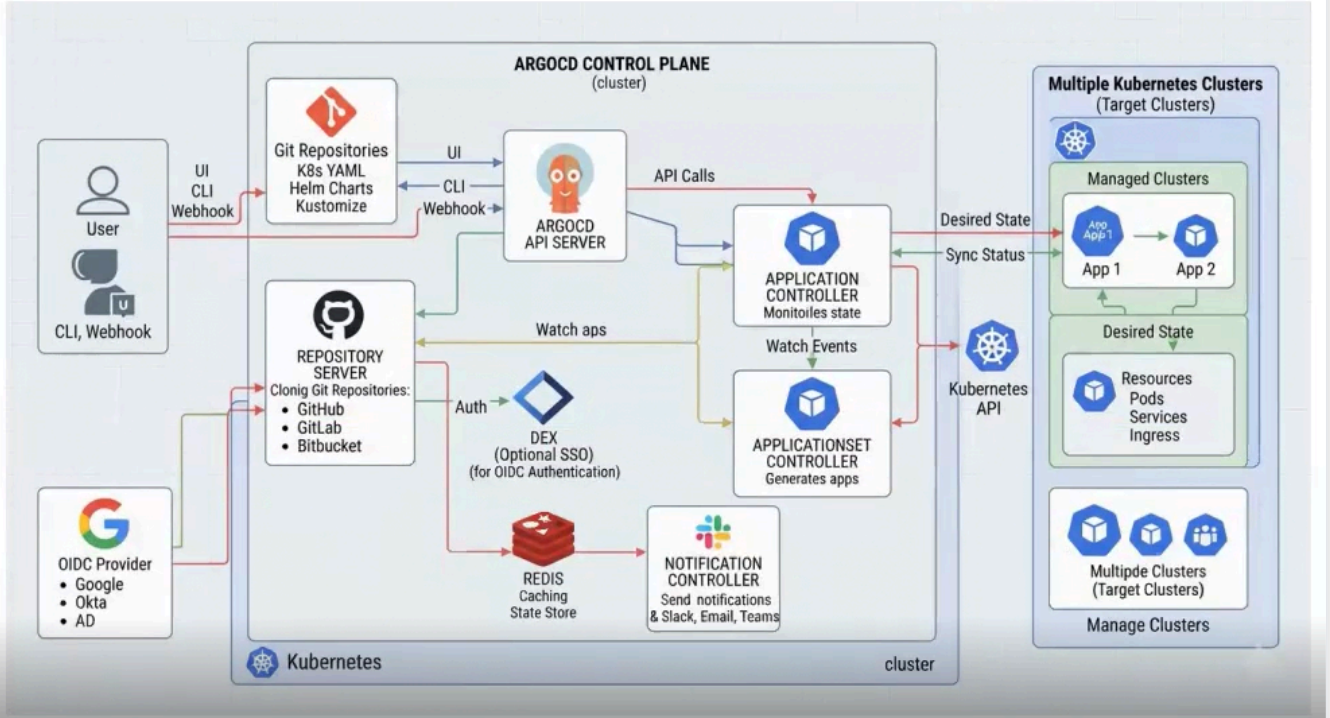
Raw screenshots captured while watching this session, kept alongside the written notes (01-argocd-components.md) for visual reference during interview revision.

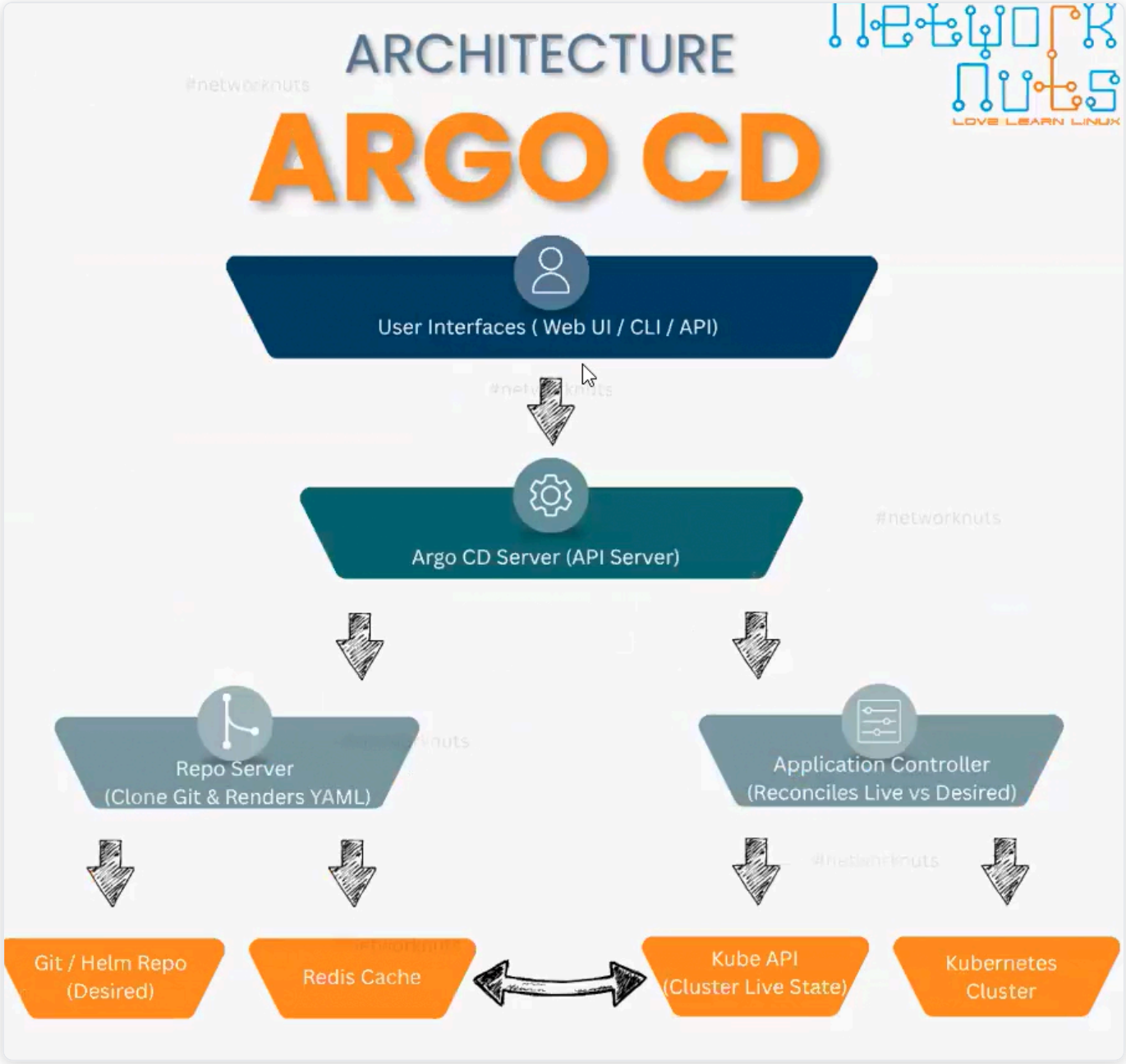
27 screenshots · captured 2026-09-11, 8:36 PM - 9:58 PM

Argo CD Components - insight

Argo CD has the following seven key components.

1. Argocd Server (API Server)
2. Application Controller
3. ApplicationSet Controller
4. Repository Server
5. Dex (optional SSO)
6. Redis
7. Notification Controller




ARCHITECTURE
ARGO CD
Network Nuts
LOVE LEARN LINUX
User Interfaces (Web UI / CLI / API)
Argo CD Server (API Server)
Repo Server
(Clone Git & Renders YAML)
Application Controller
(Reconciles Live vs Desired)
Git / Helm Repo
(Desired)
Redis Cache
Kube API
(Cluster Live State)
Kubernetes
Cluster

Argo CD Server

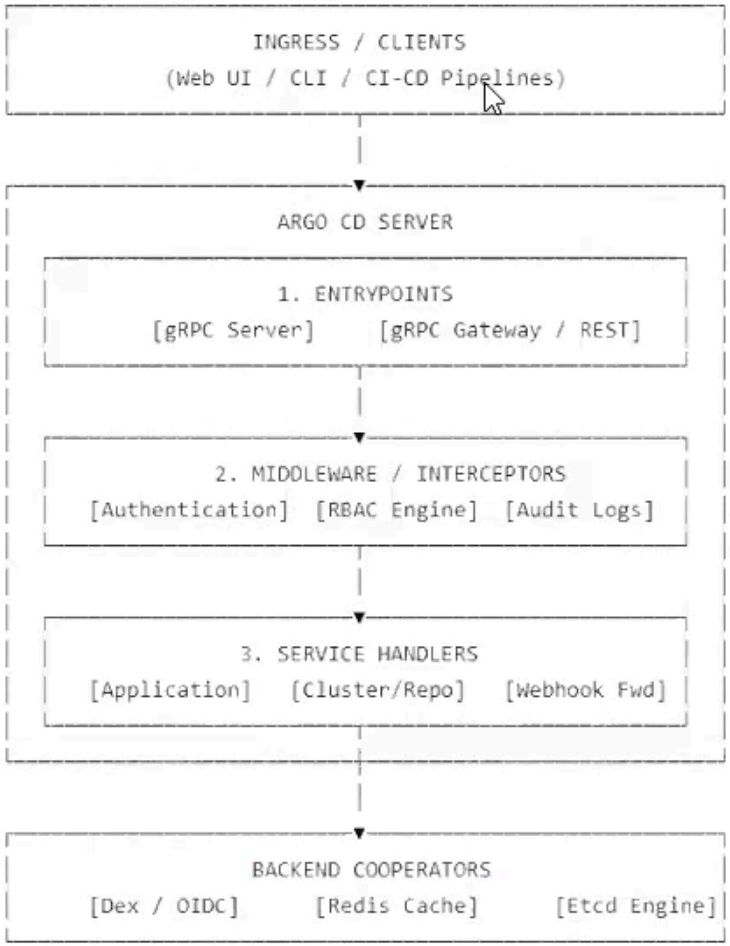
This is the API server of Argo CD that exposes Argo CD REST APIs. Its the front door to ArgoCD. It exposes three interfaces:

- A gRPC API (used by the ArgoCD CLI)
- HTTP/HTTPS endpoints for the web UI and integrations
- The web UI itself (served as static files)

The API server handles authentication and authorization. It validates JWT tokens, checks RBAC policies, and proxies requests to the appropriate backend. Accepting GitOps operations like Sync, Rollback, and App Creation. It does not do the heavy lifting of syncing or manifest generation - it delegates those tasks to other components.

All communication from UI and CLI first lands on the API server before being relayed internally.

The internal architecture of the API Server consists of three main operational tiers



Argo CD Server

1. Network Interfaces & Dual-Protocol Engine
   - gRPC Server Engine: Handled via standard HTTP/2 streams. This takes traffic by the Argo CD CLI & programmatic pipeline workers.
   - gRPC-Gateway Engine: Translates inbound JSON/REST calls over HTTP/1.1 directly into structural gRPC protocol formats inside the runtime process. This allows the Angular/React-based Web UI to interact with the underlying API using standard browser web requests without needing unique protocol proxies.

2. Interceptors & Security Middleware
   - Authentication Handler: Intercepts inbound headers to process JSON Web Tokens (JWT). If single sign-on is active, it coordinates session signatures downstream with Dex or external OpenID Connect (OIDC) brokers.
   - RBAC Enforcement Engine: Validates the payload claims against custom access security manifests declared via the cluster's global argocd-rbac-cm ConfigMap. Is user authorised to do request a action.

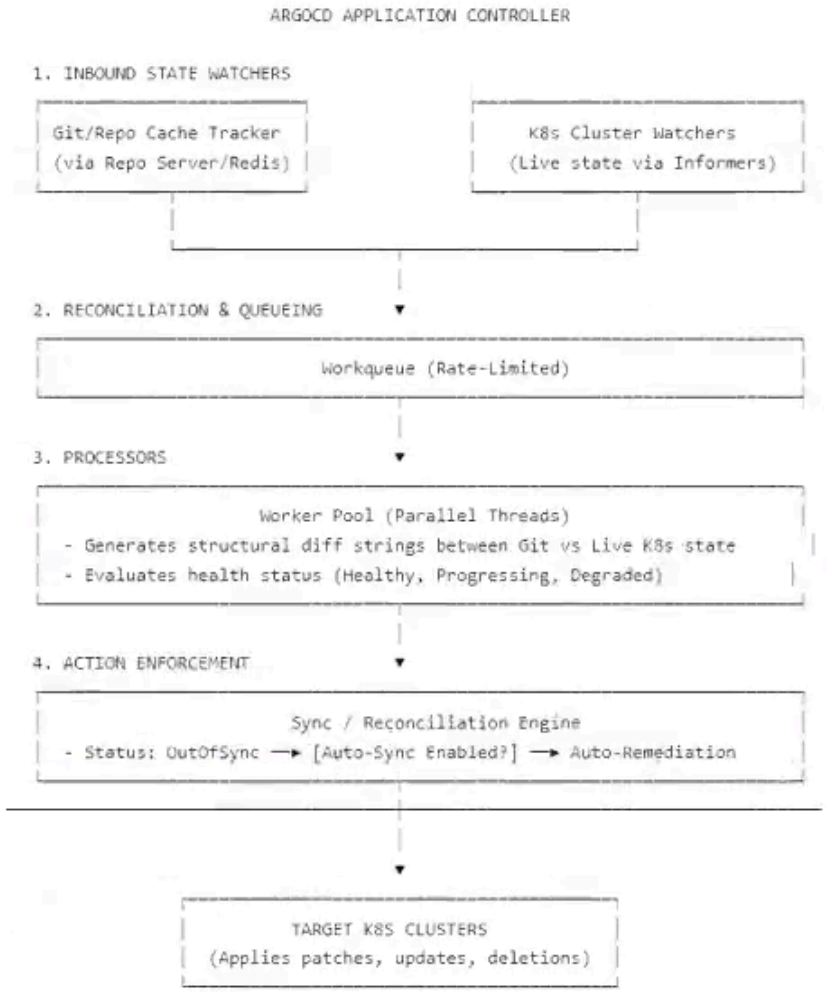
Argo CD Components - insight

Argo CD Application Controller

Application Controller is the core component of Argo CD and a Kubernetes controller, which deploys and manages the application. Its key roles are to deploy and continuously monitor the Applications created by it and compare their live state with the desired state in Git.

It is a stateful, multi-threaded Go-based Kubernetes controller that runs a continuous reconciliation loop. Its sole responsibility is to compare the live state of your Kubernetes clusters against the target state defined in your Git repositories, and eliminate any configuration drift.

Argo CD Application Controller



Argo CD Repo Server - Internals

Its the internal utility engine of Argo CD. It functions as a stateless manifest generation service. It acts as a middleman between your Git/Helm repositories and the ArgoCD Application Controller.

It handles three key tasks:

1. Cloning repositories - pulls source code from Git
2. Generating manifests - runs Helm template, Kustomize build, or plain manifest rendering
3. Caching results - stores generated manifests so it does not regenerate them on every request

The Repo server polls Git every 3 minutes by default to check for new changes, and you can also configure a webhook for Git.

Argo CD Components - insight

Argo CD Dex Server (optional) - Internals

Its an optional, bundled identity service processor that acts as an OIDC (OpenID Connect) identity provider proxy. It allows users to log into the ArgoCD UI and CLI using external enterprise identity providers (like GitHub, Google, Okta, Microsoft Entra ID/Azure AD, or OIDC/SAML providers) without ArgoCD needing to natively implement each unique authentication protocol.

Dex is packaged by default as a distinct Kubernetes deployment within standard ArgoCD installations. Here is how its architectural components interact:

- In-Memory / Shared Storage: Dex relies on a lightweight SQL database or standard Kubernetes Custom Resource Definitions (CRDs) as its backing store to manage temporary state, sign-in sessions, and keys.
- Connectors Engine: This is the core configuration layer inside Dex. It houses pluggable modules ("connectors") customized for specific providers (e.g., the github connector, saml connector, or oidc connector).
- Token Signer Engine: Once an external provider successfully authenticates a user, this module generates a cryptographically signed JSON Web Token (JWT) that ArgoCD can inherently trust.

Argo CD Redis Server - Internals

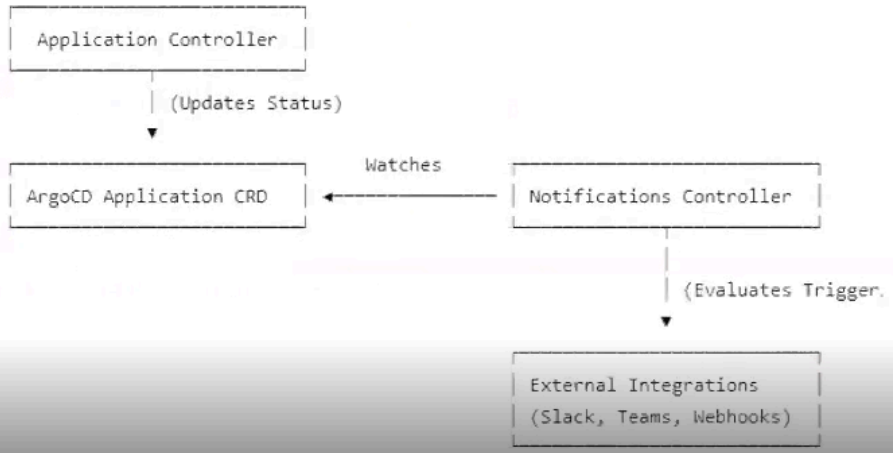
Argocd-redis is an internal, in-memory data store cache. It does not store any persistent cluster configuration or application states—those live entirely in Git and Kubernetes Custom Resources (CRDs). Instead, Redis acts purely as a performance accelerator to prevent ArgoCD from overloading external Git providers and crashing under heavy API workloads. Its core jobs are:

- Manifest Caching: Generating Kubernetes YAML from Helm charts or Kustomize files is CPU and time-intensive. Redis caches these compiled manifests. If a commit hash has not changed, ArgoCD pulls the YAML directly from Redis in milliseconds instead of recalculating it.
- Git Revision Checking: It caches recent Git repository commit SHAs, tags, and branch tracking data to speed up application status checks.
- OIDC/Dex Session State: It caches temporary OAuth/OIDC state data, single sign-on (SSO) tokens, and authentication sessions handled by the API server or Dex.
- API Rate-Limit Protection: By serving repetitive user queries and automated reconciliation loops directly from memory, Redis stops ArgoCD from hitting API rate limits on platforms like GitHub or GitLab.

Argo CD Components - insight

Argo CD Notifications Controller - Internals

Dedicated microservice that continuously watches your application states and dispatches real-time alerts to external communication channels (like Slack, Microsoft Teams, Email, Telegram, or Webhooks) when specific deployment events occur.



Managing Clusters with Argo CD

GitOps becomes powerful when you scale it across environments — dev, staging, production.

Argo CD works as a central control plane that can manage multiple Kubernetes clusters at once.

In-Cluster vs External Clusters

If Argo CD runs inside the same cluster it manages, it's called in-cluster management.

When it connects to other Kubernetes clusters outside of its own—across clouds or regions—that's external cluster management.

Many teams deploy Argo CD in a secure staging cluster and connect it externally to production, development, and QA clusters.

Managing Clusters with Argo CD

Sync Strategies

- Manual: It requires manual intervention. For example, you have to click "Sync" in the Argo CD UI or approve the change via a CI/CD pipeline.
- Automatic: Argo CD applies Git changes instantly. This feature simplifies testing during development, allowing you to make frequent changes easily.
- Selective: You can decide which apps auto-sync and which need manual approval. For example, lower stage environments are suitable for automatic sync, while higher environments like pre-prod or prod require human review and approval. You can control the deployment behavior per app or per project.

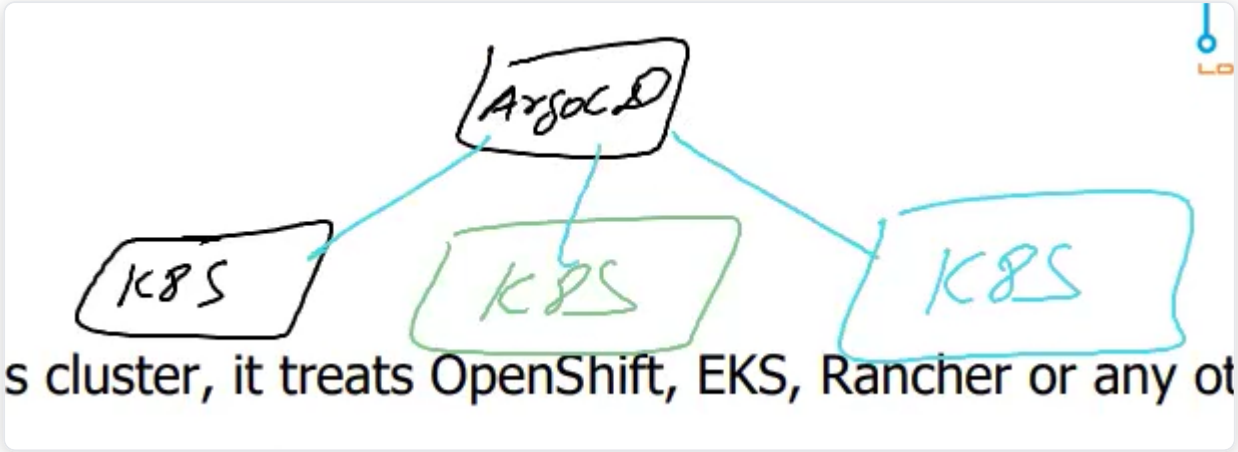
Argo CD Installation Types

Vanilla Argo CD vs Enterprise Distributions

Feature	Vanilla ArgoCD	Red Hat OpenShift GitOps	Akamai/Akuity Enterprise
Core Engine	Pure upstream ArgoCD	Pure upstream ArgoCD	Pure upstream ArgoCD
Installation	Manual YAML / Helm	OpenShift Operator Hub	Managed SaaS Control Plane
Authentication	Dex / Manual OIDC	Integrated with OpenShift OAuth	Enhanced Multi-Tenant SSO
Management	You manage infrastructure	Red Hat manages cluster platform	Akuity manages instances (SaaS)
Support	Community Slack / GitHub Issues	Enterprise SLA by Red Hat	Enterprise SLA by Akuity (built by Argo creators)

We can use vanilla ArgoCD architecture in two ways to manage these platforms:

- Hub-and-Spoke (Recommended): Install vanilla ArgoCD on a centralized management cluster. From that single dashboard, register your external OpenShift and Rancher downstream clusters via secure network endpoints. ArgoCD will deploy workloads remotely.
- In-Cluster App (Local): Deploy a completely separate instance of vanilla ArgoCD directly inside a specific OpenShift project or a Rancher-managed cluster to handle only local workloads.


ArgoCD
K8S
K8S
K8S
s cluster, it treats OpenShift, EKS, Rancher or any ot

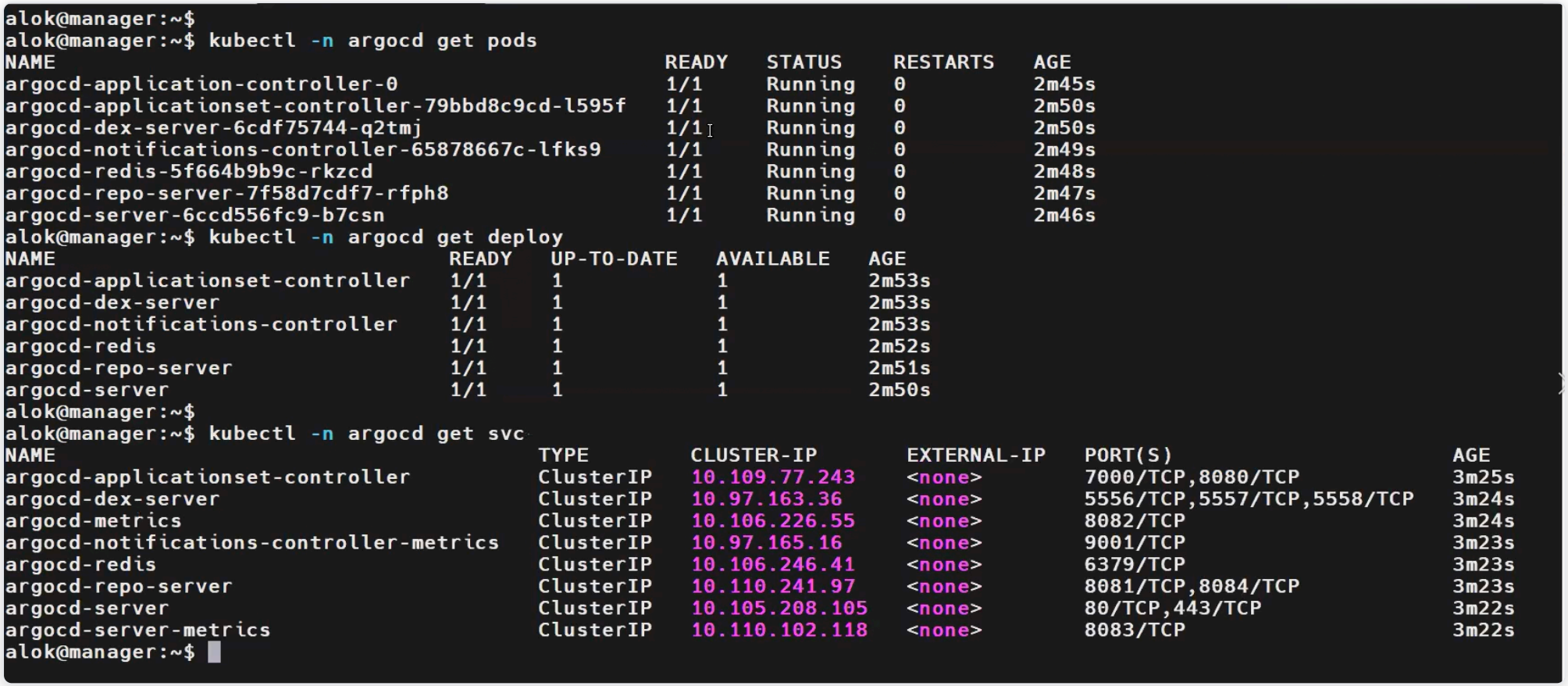
Argo CD Installation Types

Next decision is the scope that Argo CD should manage. By default, Argo CD has the authority to control resources across an entire Kubernetes cluster. This is the preferred option, especially when Argo CD is being used by Kubernetes cluster administrators. However, another approach, known as "namespaced" mode, that is available is to deploy Argo CD within a specific namespace and to allow Argo CD to only manage resources within specific namespaces.

kubectl create namespace argocd

kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

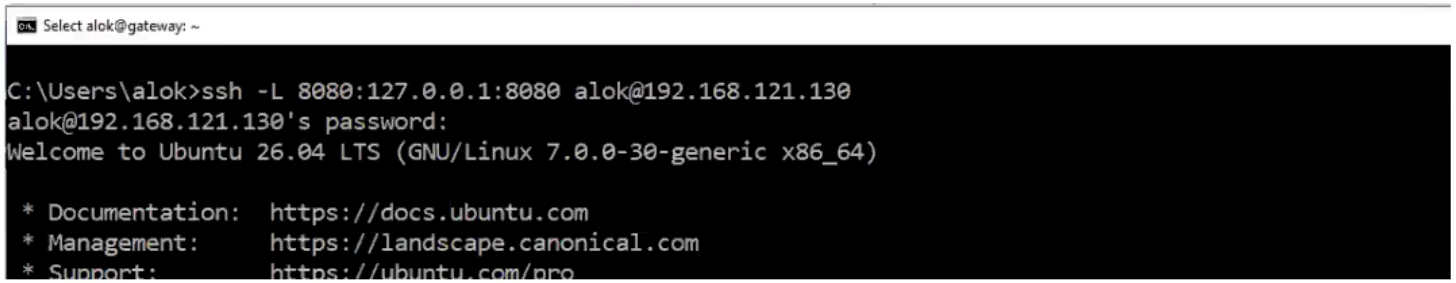
```
alok@manager01:~$
alok@manager01:~$ kubectl -n argocd get deploy
NAME                               READY   UP-TO-DATE   AVAILABLE   AGE
argocd-applicationset-controller   1/1     1            1           2m38s
argocd-dex-server                  1/1     1            1           2m38s
argocd-notifications-controller    1/1     1            1           2m38s
argocd-redis                       1/1     1            1           2m38s
argocd-repo-server                 1/1     1            1           2m38s
argocd-server                      1/1     1            1           2m38s
alok@manager01:~$
alok@manager01:~$ kubectl -n argocd get svc
NAME                                      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
argocd-applicationset-controller          ClusterIP   10.109.235.37    <none>        7000/TCP,8080/TCP
argocd-dex-server                         ClusterIP   10.104.19.113    <none>        5556/TCP,5557/TCP,5558/TCP
argocd-metrics                            ClusterIP   10.108.53.116    <none>        8082/TCP
argocd-notifications-controller-metrics   ClusterIP   10.107.53.113    <none>        9001/TCP
argocd-redis                              ClusterIP   10.96.208.60     <none>        6379/TCP
argocd-repo-server                        ClusterIP   10.97.163.163    <none>        8081/TCP,8084/TCP
argocd-server                             ClusterIP   10.102.242.89    <none>        80/TCP,443/TCP
argocd-server-metrics                     ClusterIP   10.101.216.147   <none>        8083/TCP
alok@manager01:~$
```


alok@manager:~$
alok@manager:~$ kubectl -n argocd get pods
NAME READY STATUS RESTARTS AGE
argocd-application-controller-0 1/1 Running 0 2m45s
argocd-applicationset-controller-79bbd8c9cd-l595f 1/1 Running 0 2m50s
argocd-dex-server-6cdf75744-q2tmj 1/1 Running 0 2m50s
argocd-notifications-controller-65878667c-lfks9 1/1 Running 0 2m49s
argocd-redis-5f664b9b9c-rkzcd 1/1 Running 0 2m48s
argocd-repo-server-7f58d7cdf7-rfph8 1/1 Running 0 2m47s
argocd-server-6ccd556fc9-b7csn 1/1 Running 0 2m46s
alok@manager:~$ kubectl -n argocd get deploy
NAME READY UP-TO-DATE AVAILABLE AGE
argocd-applicationset-controller 1/1 1 1 2m53s
argocd-dex-server 1/1 1 1 2m53s
argocd-notifications-controller 1/1 1 1 2m53s
argocd-redis 1/1 1 1 2m52s
argocd-repo-server 1/1 1 1 2m51s
argocd-server 1/1 1 1 2m50s
alok@manager:~$
alok@manager:~$ kubectl -n argocd get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
argocd-applicationset-controller ClusterIP 10.109.77.243 <none> 7000/TCP,8080/TCP 3m25s
argocd-dex-server ClusterIP 10.97.163.36 <none> 5556/TCP,5557/TCP,5558/TCP 3m24s
argocd-metrics ClusterIP 10.106.226.55 <none> 8082/TCP 3m24s
argocd-notifications-controller-metrics ClusterIP 10.97.165.16 <none> 9001/TCP 3m23s
argocd-redis ClusterIP 10.106.246.41 <none> 6379/TCP 3m23s
argocd-repo-server ClusterIP 10.110.241.97 <none> 8081/TCP,8084/TCP 3m23s
argocd-server ClusterIP 10.105.208.105 <none> 80/TCP,443/TCP 3m22s
argocd-server-metrics ClusterIP 10.110.102.118 <none> 8083/TCP 3m22s
alok@manager:~$

Installing Argo CD using manifest

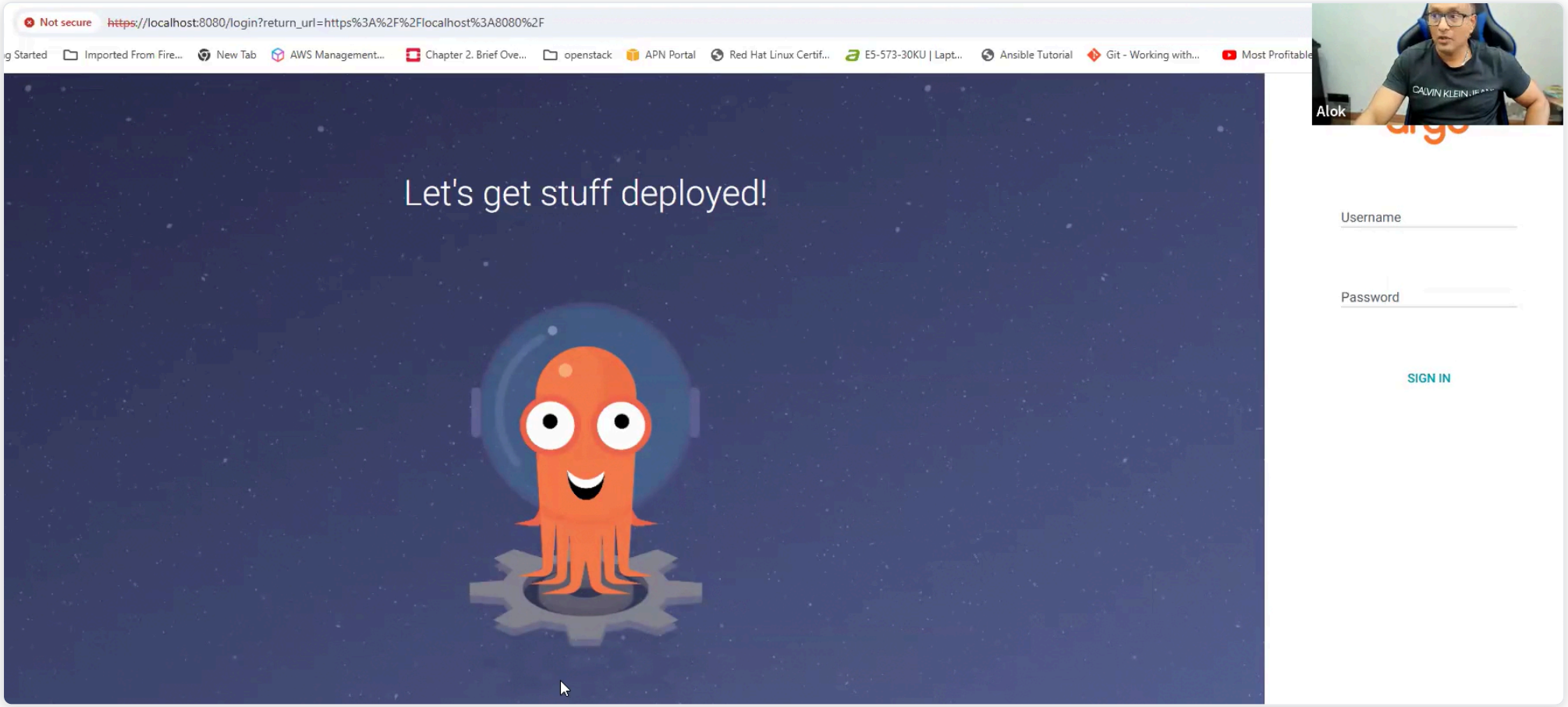
Accessing Argo CD UI from your localhost (Laptop / Deskstop)

#1 - Establish an SSH tunnel from your local laptop to the gateway server, opening up port 8080:
ssh -L 8080:127.0.0.1:8080 alok@192.168.121.130 (my gateway ip)

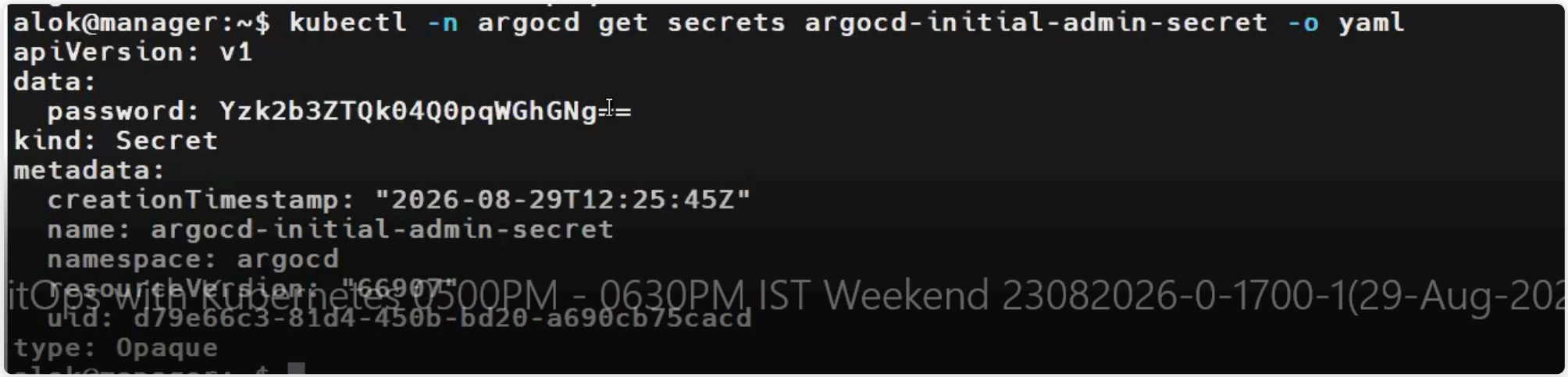


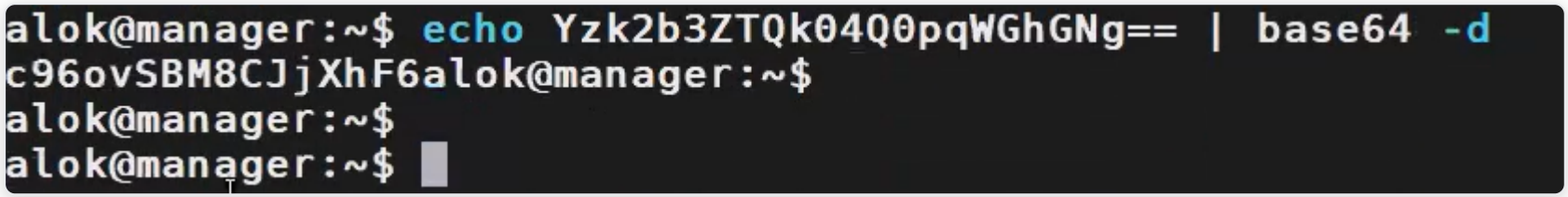
#2 - Open a terminal session inside the gateway server and start a live Kubernetes port forward to the cluster pod:
kubectl port-forward svc/argocd-server -n argocd 8080:443

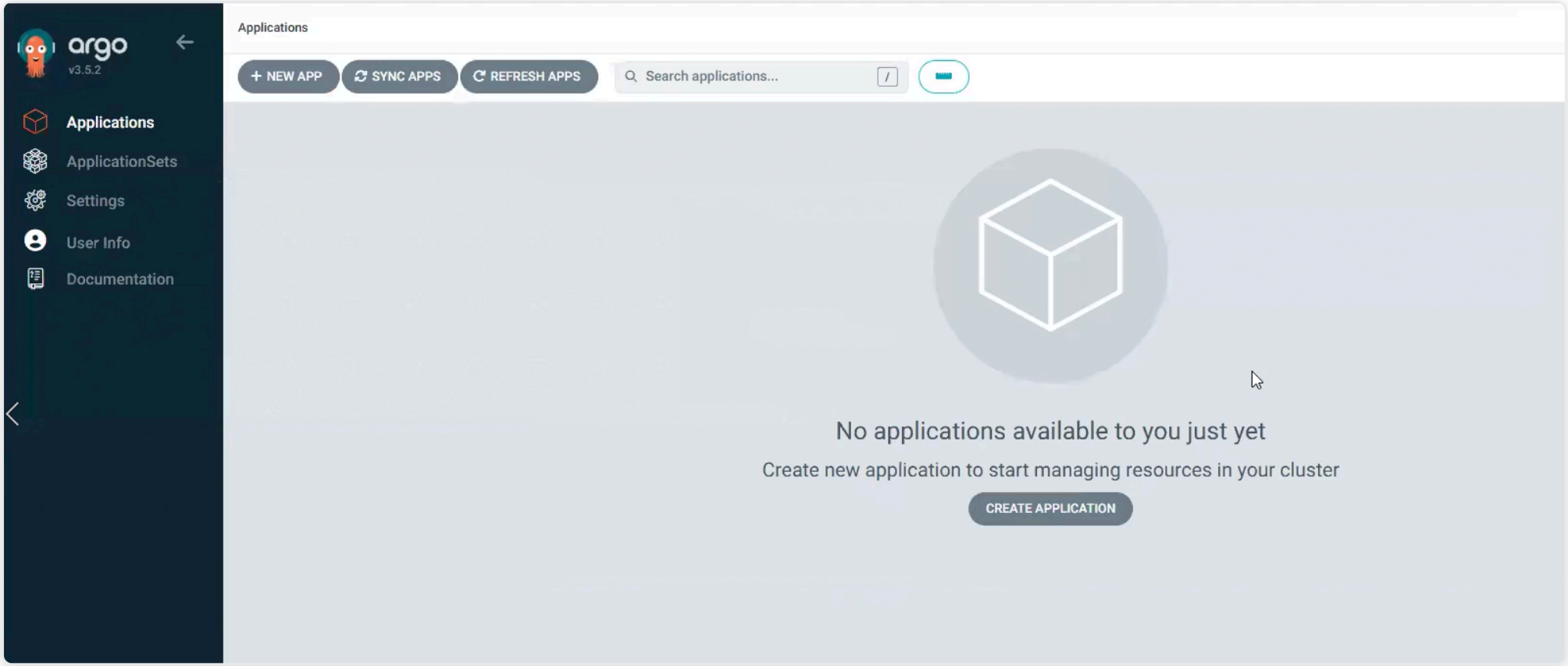



Not secure
https://localhost:8080/login?return_url=https%3A%2F%2Flocalhost%3A8080%2F
Started
Imported From Fire...
New Tab
AWS Management...
Chapter 2. Brief Ove...
openstack
APN Portal
Red Hat Linux Certif...
E5-573-30KU | Lapt...
Ansible Tutorial
Git - Working with...
Most Profitable
Alok
Let's get stuff deployed!
Username
Password
SIGN IN

```
alok@manager:~$ kubectl -n argocd get secrets
NAME                              TYPE     DATA   AGE
argocd-initial-admin-secret       Opaque   1      7m1s
argocd-notifications-secret       Opaque   0      9m20s
argocd-redis                      Opaque   1      7m5s
argocd-secret                     Opaque   5      9m20s
alok@manager:~$
```


alok@manager:~$ kubectl -n argocd get secrets argocd-initial-admin-secret -o yaml
apiVersion: v1
data:
password: Yzk2b3ZTQk04Q0pqWGhGNg==
kind: Secret
metadata:
creationTimestamp: "2026-08-29T12:25:45Z"
name: argocd-initial-admin-secret
namespace: argocd
resourceVersion: "66907"
uid: d79e66c3-81d4-450b-bd20-a690cb75cacd
type: Opaque

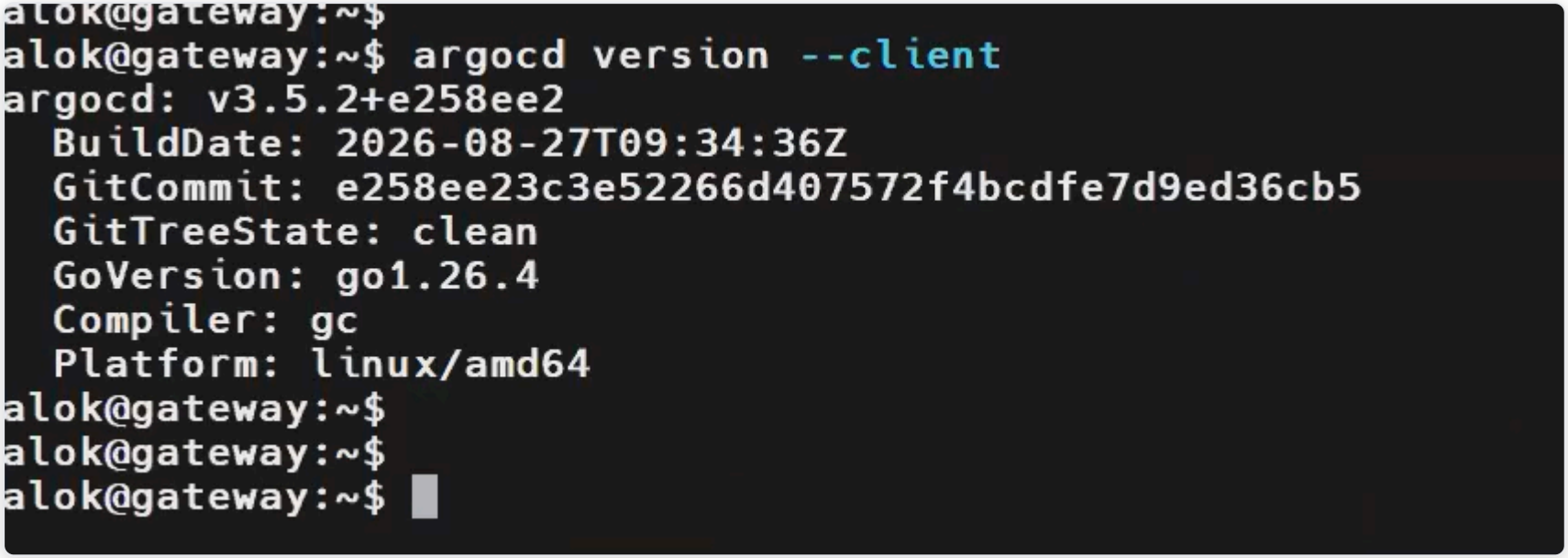

alok@manager:~$ echo Yzk2b3ZTQk04Q0pqWGhGNg== | base64 -d
c96ovSBM8CJjXhF6alok@manager:~$
alok@manager:~$
alok@manager:~$


argo
v3.5.2
Applications
ApplicationSets
Settings
User Info
Documentation
Applications
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
/
No applications available to you just yet
Create new application to start managing resources in your cluster
CREATE APPLICATION

Installing Argo CD CLI

The Argo CD CLI (argocd) gives you full control over your GitOps workflows from the terminal. It's a great tool for automation and for users who prefer command-line tools over the UI.

```
# 1. Download the latest binary
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
# 2. Install it with executable permissions to your system path
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
# 3. Clean up the downloaded installer file
rm argocd-linux-amd64
# 4. Verify
argocd version --client
```


alok@gateway:~$
alok@gateway:~$ argocd version --client
argocd: v3.5.2+e258ee2
BuildDate: 2026-08-27T09:34:36Z
GitCommit: e258ee23c3e52266d407572f4bcdfe7d9ed36cb5
GitTreeState: clean
GoVersion: go1.26.4
Compiler: gc
Platform: linux/amd64
alok@gateway:~$
alok@gateway:~$
alok@gateway:~$